

1.1 全局关系（维持原文 5-fold）

has_binding_to、upregulates、downregulates、ppi、miRNA_regulates、tf_regulates、rbp_regulates、lmi、lncRNA_regulates、transcribes、translates、participates_in 保持原有切分方式

1.2 少样本关系：Entity-Level 5-Fold CV（按 head 实体切分）

1.1 is_subtype_of（48 条）：

按 head 实体（48 个亚型）分 5 组（10/10/10/10/8）。Fold k 时，Group k 的亚型作为验证 head，训练集删除头实体是group k里面的亚型、关系是is_subtype_of的所有三元组这些亚型的，但保留这些亚型在subdisease_regulates 中的 triple。

1.2 subdisease_regulates（193 条）：按 tail 实体（173 个基因）分 5 组（35/35/35/35/33）。Fold k 时，训练集删除所有 tail 为fold k里面的基因的、关系是 subdisease_regulates的所有 triple。

1.3 同步规则：两套 fold 同步切换（Fold 1 同时验证的验证集中出现 is_subtype_of 的 Group 1 亚型和 subdisease_regulates 的 Group 1 基因）

1.4 验证集污染检查：训练前运行脚本，确保：

- is_subtype_of 验证集中的 head 亚型，is_subtype_of 为关系的三元组在训练集中未出现
- subdisease_regulates 验证集中的 tail 基因，subdisease_regulates 为关系的三元组在训练集中未出现

Python

```
def check_validation_leakage(train_triples, valid_triples, relation_name):
    """检查验证集 head 实体是否在训练集同关系中出现过"""
    train_heads = set(h for h, r, t in train_triples if r == relation_name)
    valid_heads = set(h for h, r, t in valid_triples if r == relation_name)
    leakage = train_heads & valid_heads
    assert len(leakage) == 0, \
        f"[{relation_name}] 验证集污染！{len(leakage)} 个 head 实体在训练集出现：{list(leakage)[:5]}"

    # 同时检查 exact triple 泄露
    train_set = set(train_triples)
```

```
valid_set = set(valid_triples)
triple_leak = train_set & valid_set
assert len(triple_leak) == 0, f"[{relation_name}] 存在 {len(triple_leak)} 条
exact triple 泄露”
# 训练前调用
check_validation_leakage(train, valid, 'is_subtype_of')
check_validation_leakage(train, valid, 'subdisease_regulates')
```